Temporal

CASE STUDY · NOVEMBER 2026

Paying exactly once: moving an AI agent's refund approvals onto Temporal and Nexus

How a support team can let an LLM agent propose refunds, keep a human in the loop for as long as the approval takes, and hand the money movement to the payments team behind a contract, without paying anyone twice.

10 of 22

paid exactly once by today's stack, under the same faults

22 of 22

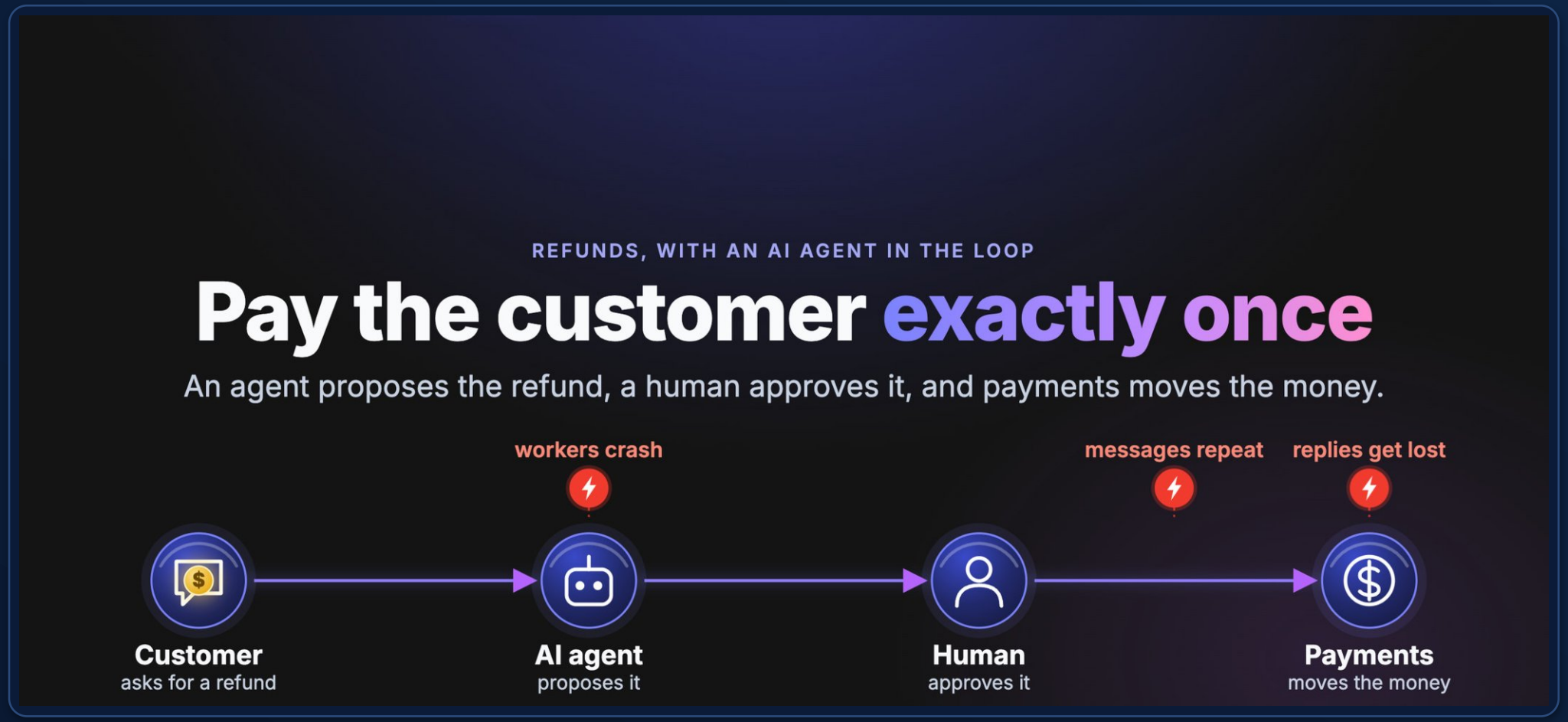
paid exactly once on Temporal + Nexus

Kevin M. Butler · github.com/agileresearchservices/paid-once

An independent case study, prepared for Temporal as part of a panel interview. It was not produced by Temporal. Temporal is a trademark of Temporal Technologies, Inc., used here to identify the product the study is about.

THE PROBLEM

A refund looks simple. Then something fails.



Same tickets. Same failures. Two builds. The payments ledger decides which one pays once.

WHO FACES THIS, AND WHY NOW

The pain lands on three teams, and the payments team holds the gate

Role	The pain
Support / AI platform	Agent runs stall or duplicate on deploys and crashes; long approvals are fragile; no clean way to prove what happened
Payments platform	Reluctant to hand an agent a raw refund API; wants a narrow, auditable contract and a contained blast radius
Finance / risk	A duplicate refund is a direct loss; needs proof that a human approved the exact action

NOT A FIT

One team in one namespace, low volume, or no money movement. Nexus adds little there; child workflows and activities already compose work inside one team's namespace.

WHY NOW

1. **Agents are moving from drafting to acting.** An approval gate is the obvious guardrail, and it puts a human wait in the middle of an agent run.
2. **Pilots stall at production readiness.** The stall is in execution (state, retries, deploys), not in the model.
3. **Payments owns its API.** So the agent team needs a contract, not direct access.

Each of these is an assumption to check against your own situation.

DEMO STEP 2 OF 9 · ARCHITECTURES

Two ways to build the same refund

An agent proposes, a human approves, payments moves the money. Built twice.

WATCH

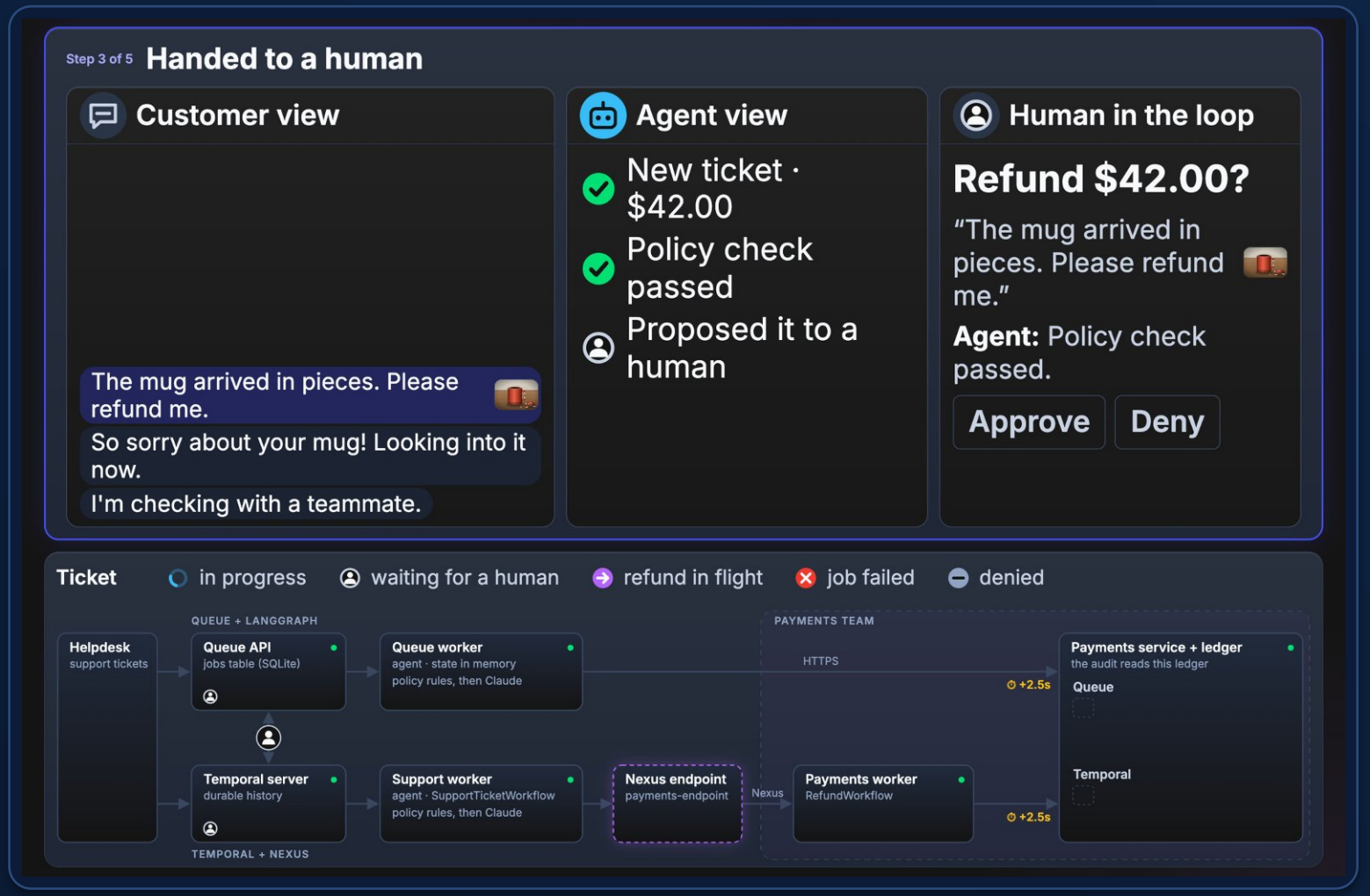
Where each stack keeps a ticket's progress.

RESULT

One ticket, refunded once in both stacks.

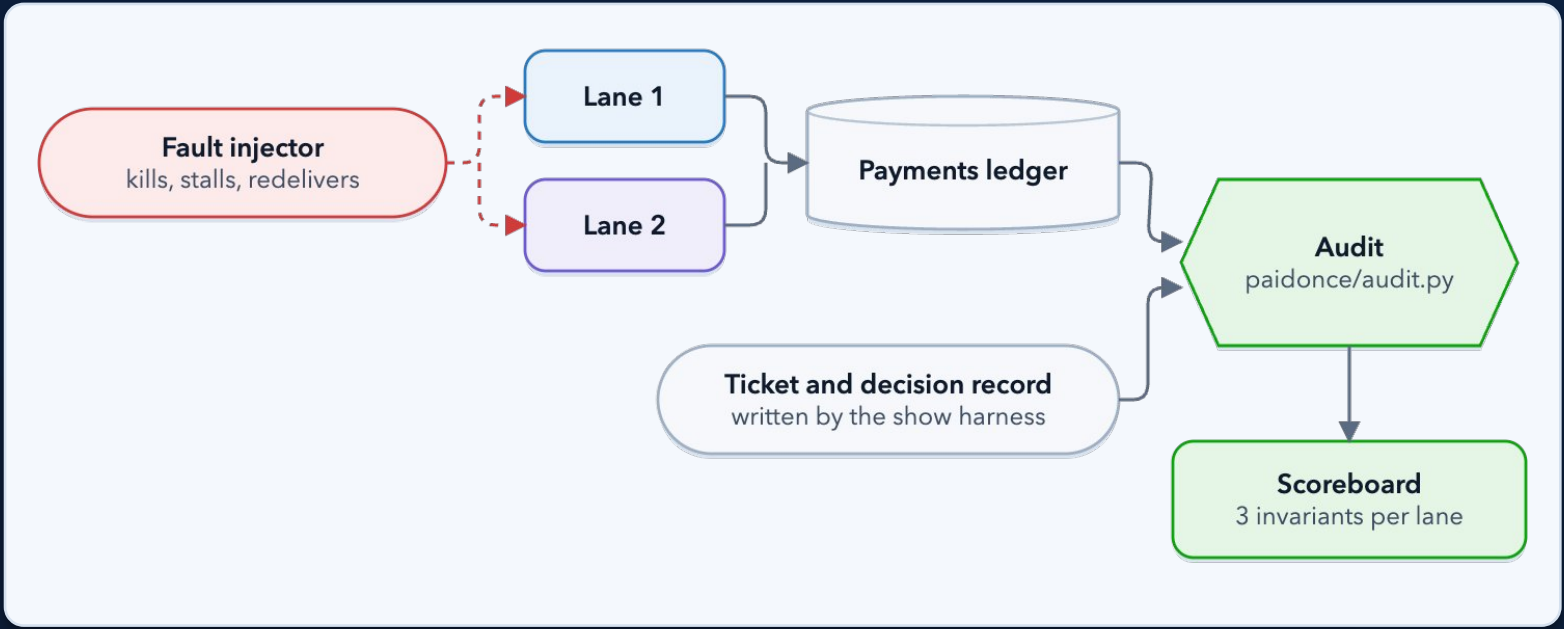
Queue + LangGraph keeps progress in a jobs table and in worker memory. Temporal + Nexus keeps it in the workflow's durable history.

▶ Click the picture to play the clip in PowerPoint or Keynote.



HOW IT WAS TESTED

Same tickets, same faults, scored from the payments ledger



The same 22 approved tickets go through each build.

A fault injector kills workers, redelivers queue messages and makes payments commit, then go quiet.

The audit reads the ledger, never a build's own records.

INVARIANT 1

Paid exactly once

Every approved ticket is refunded exactly once.

INVARIANT 2

Approved first

No refund exists without an approval.

INVARIANT 3

Nothing lost

No ticket is lost or stuck: an approval with no refund after 20 seconds counts as lost.

DEMO STEP 3 OF 9 · HAPPY PATH

Happy path: nothing goes wrong

Seven tickets per stack. The agent denies one, the human denies one, five are refunded.

WATCH

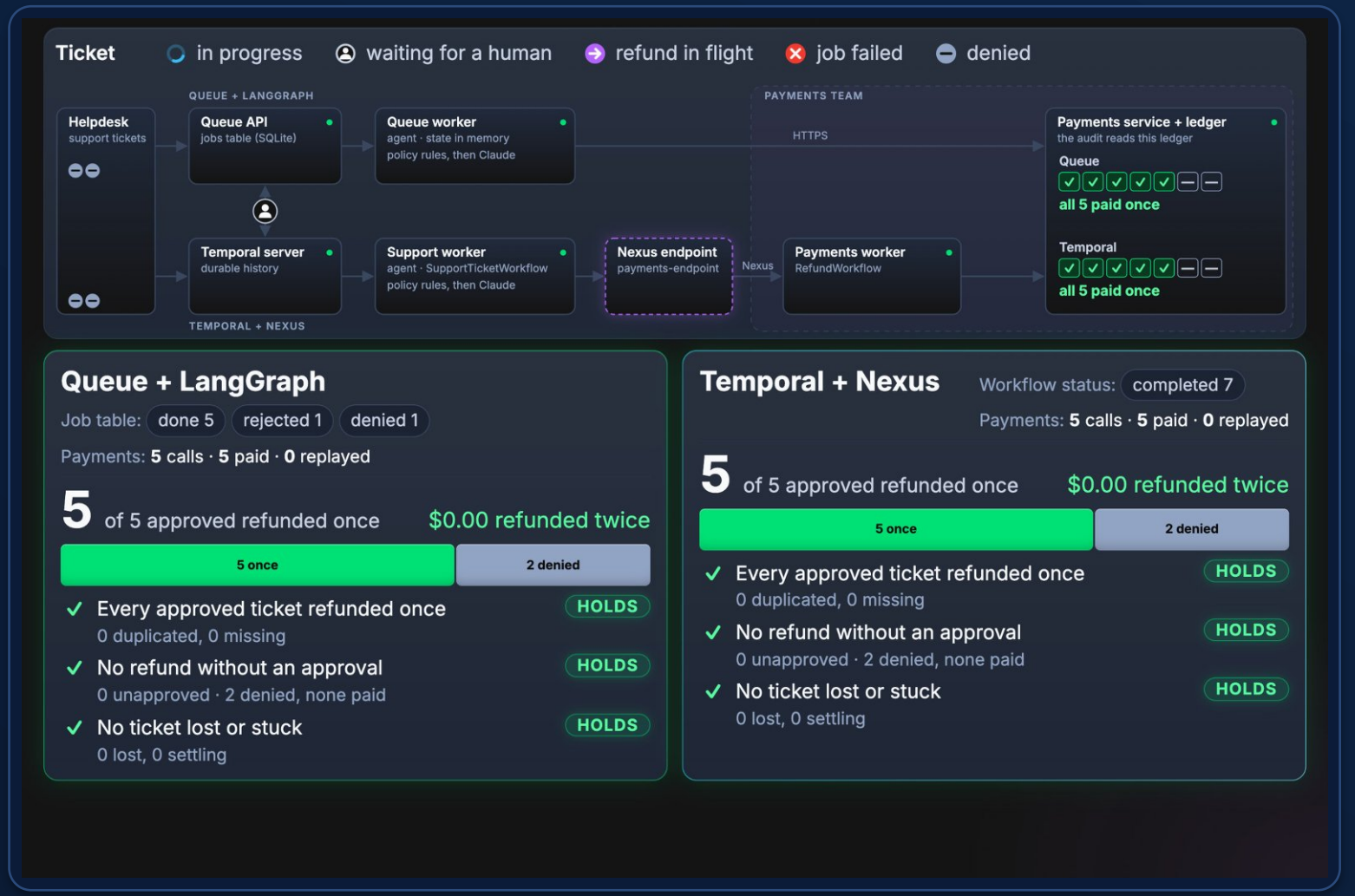
Denied tickets go back to the helpdesk.

RESULT

Both stacks: five refunded once, two denied, no money on the denials.

In this capture: both stacks refunded 5 of 5 once, with 2 denied and nothing paid on the denials. Every invariant holds in both.

▶ Click the picture to play the clip in PowerPoint or Keynote.



DEMO STEP 4 OF 9 · CRASH

Crash or deploy: the agent workers die

Three refunds are in flight, the rest wait for a human. We kill the agent workers, then give the audit 20 seconds to see which tickets never get paid.

WATCH

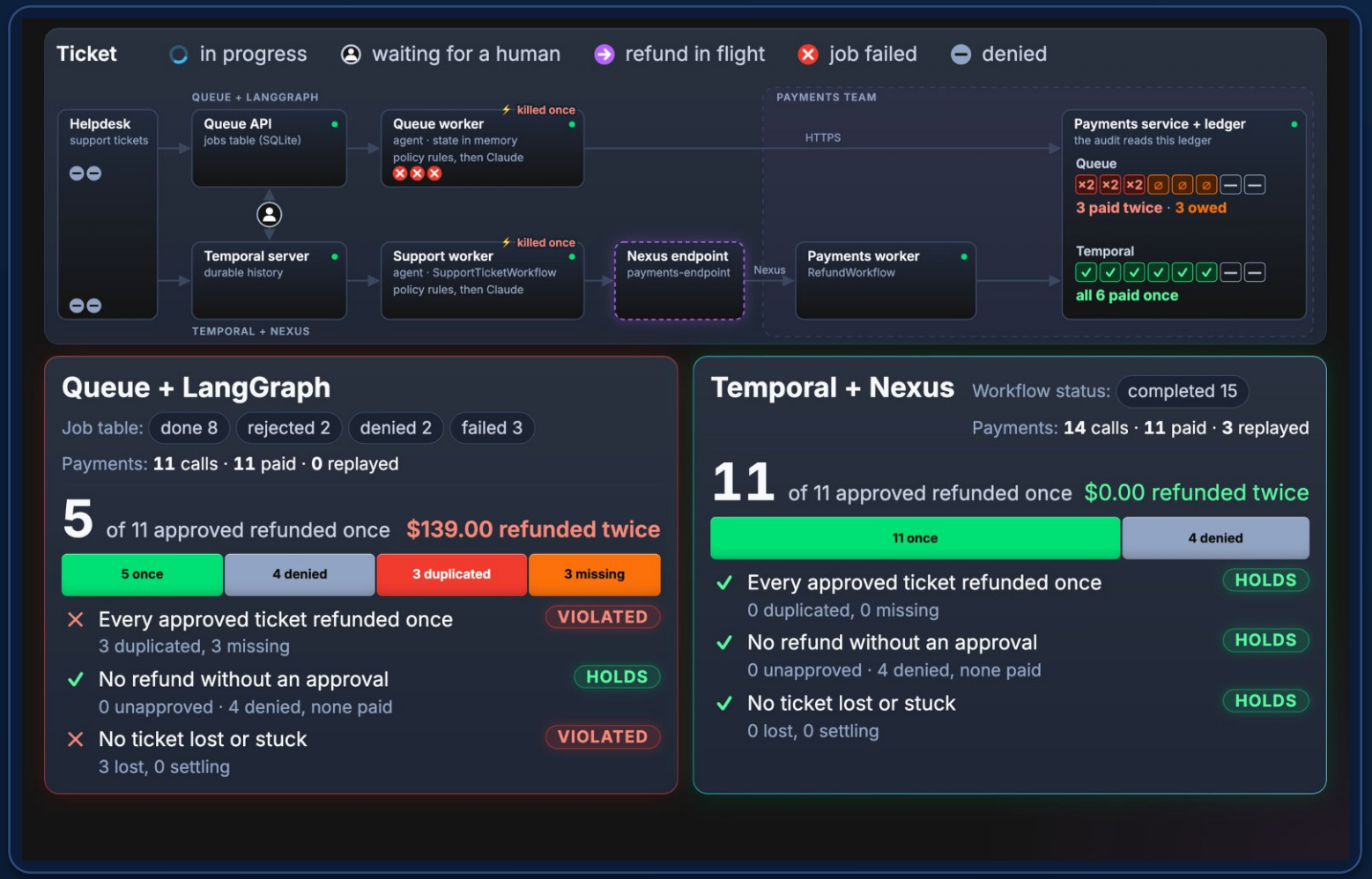
In-flight refunds paid once? Waiting tickets still refunded?

RESULT

Queue + LangGraph: paid twice, tickets lost. Temporal: paid once, nothing lost.

In this capture, this scene: Queue + LangGraph paid 3 tickets twice and never paid 3 approvals. Temporal paid all 6 once. The idle 20-second audit wait is sped up in the clip.

▶ Click the picture to play the clip in PowerPoint or Keynote.



DEMO STEP 5 OF 9 · REDELIVERY

The same message arrives twice

The queue redelivers jobs. Temporal gets each approval again.

WATCH

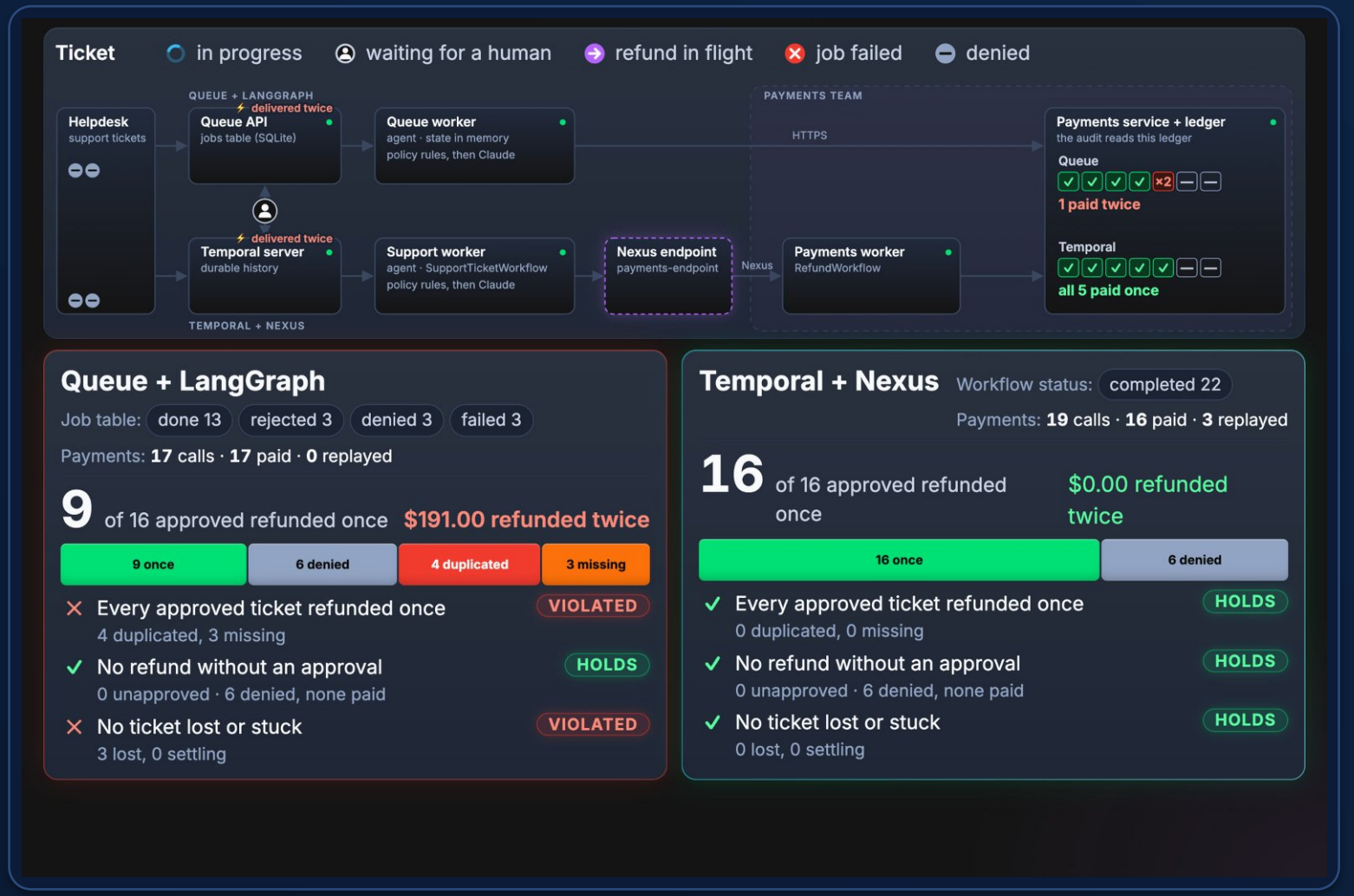
Dollars refunded twice.

RESULT

Queue + LangGraph: some refunded twice. Temporal: the first decision wins.

In this capture, this scene: Queue + LangGraph paid 1 ticket twice and 4 once. Temporal paid all 5 once.

▶ Click the picture to play the clip in PowerPoint or Keynote.



DEMO STEP 6 OF 9 · PAYMENTS QUIET

Payments pays it, then goes quiet

Payments saves the refund, but the reply never arrives. Both stacks retry.

WATCH

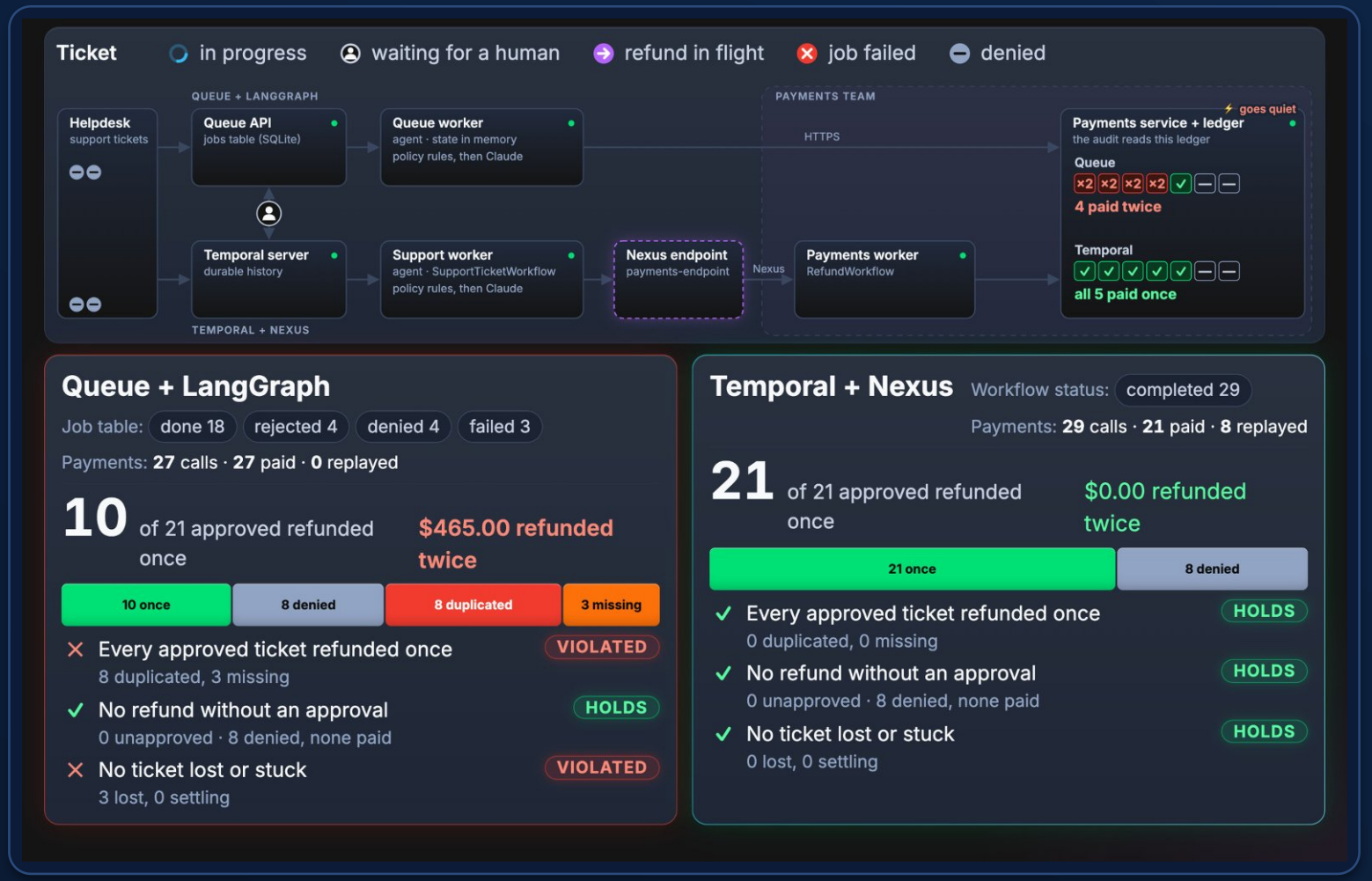
Which retries does the ledger recognise?

RESULT

Temporal reuses the ticket's key: answered from the ledger. Queue + LangGraph sends a new key: paid again.

In this capture, this scene: Queue + LangGraph paid 4 tickets twice and 1 once. Temporal paid all 5 once, and each retry was answered from the ledger (5 replays in this scene).

▶ Click the picture to play the clip in PowerPoint or Keynote.



DEMO STEP 7 OF 9 · SCOREBOARD

The ledger has the final word



Capture of one run. The on-screen scoreboard leaves out the walkthrough ticket, so it reads 21; with it the study counts 22. Counts vary by a ticket or two between runs.

THE TWO MEASURED RUNS, 22 APPROVED TICKETS

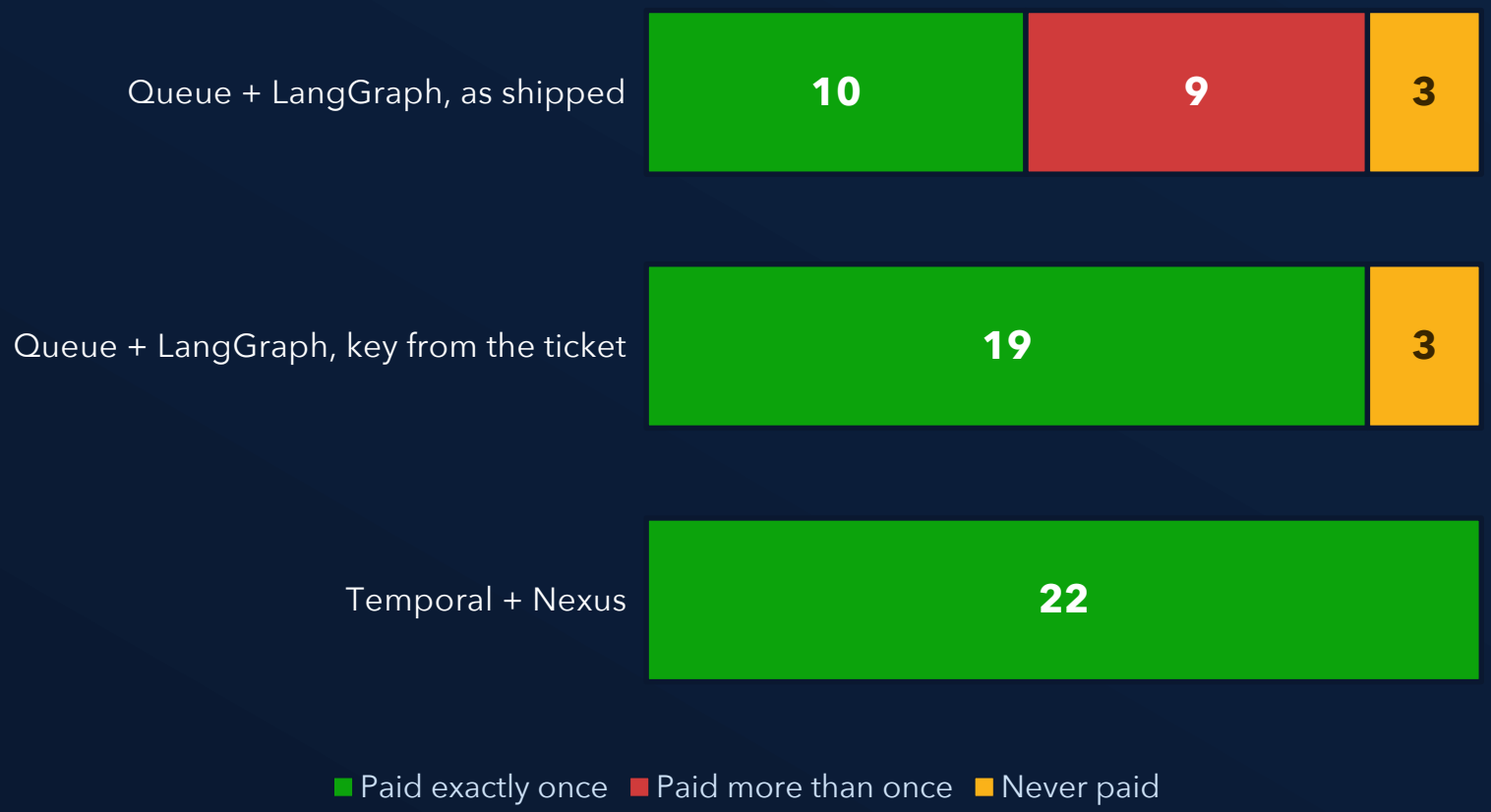
	Run 1	Run 2	Temporal
Paid exactly once	10	9	22
Paid more than once	9	10	0
Never paid	3	3	0

Queue + LangGraph as shipped: two runs on 2026-11-01, same faults, one laptop. Temporal + Nexus gave 22 of 22 in both.

Scored from the payments ledger, never from either build's own records.

RESULTS

The one-line key fix closes duplicates, not lost approvals



First of two 2026-11-01 runs per configuration. Second run of the stack as shipped: 9 once, 10 more than once, 3 never.

THE SURPRISE

A one-line change to the existing stack's key removed every duplicate.

It did not recover the 3 approvals that died with their worker.

WHAT THE PLATFORM GIVES

The approval wait survives restarts and deploys, because it lives in the workflow history, not in worker memory.

DEMO STEP 8 OF 9 · NEXUS

Nexus gives payments a contract it owns

WITHOUT NEXUS

Support has to know payments' namespace, task queue, workflow type and retry policy, and owns the client, timeouts and retries.

WITH NEXUS

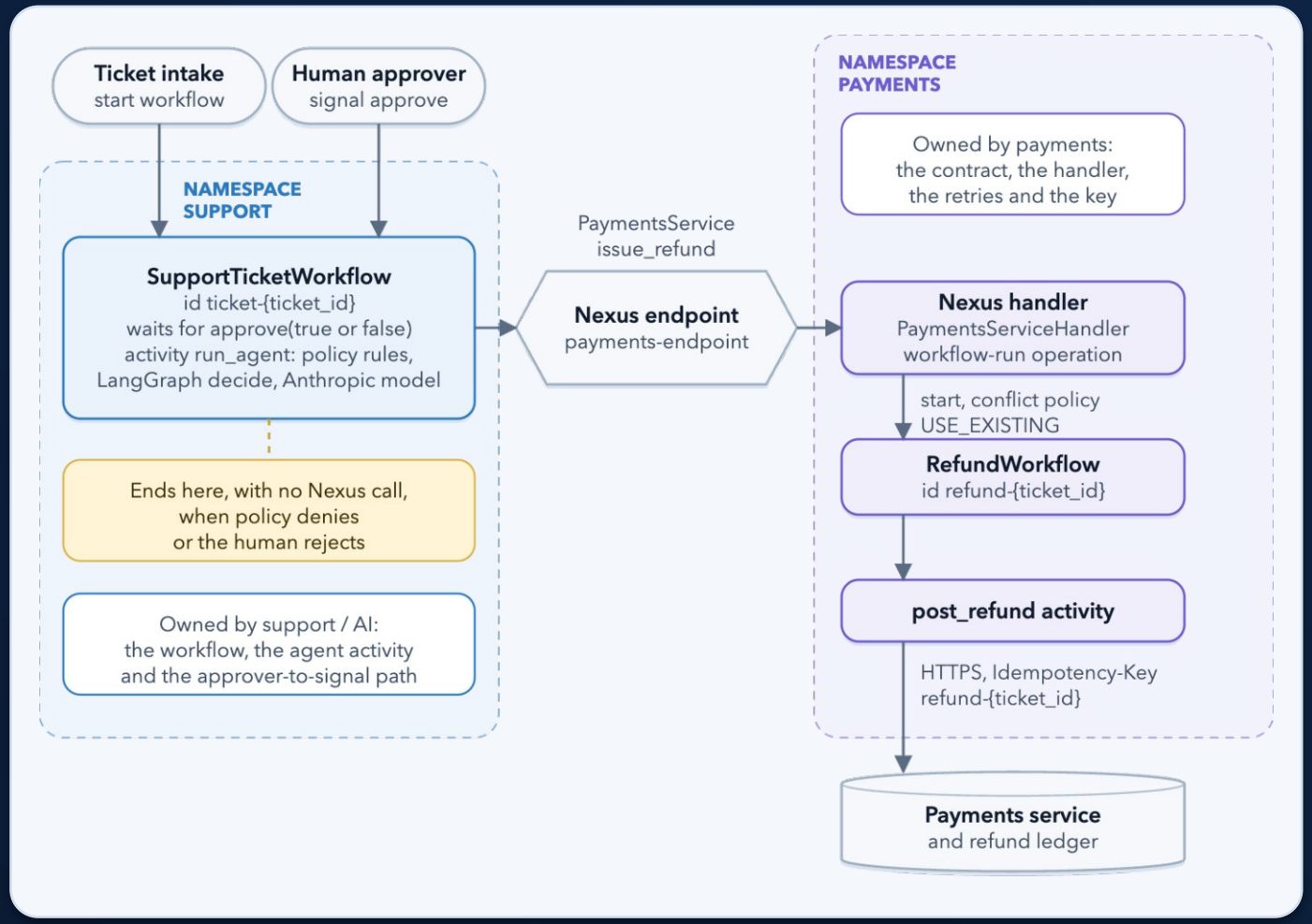
One durable call, issue_refund, to one endpoint. Payments owns everything behind it, including the key.

THE CONTRACT

Support sends `ticket_id, amount_cents`

Payments answers `refund_id, ticket_id, amount_cents`

Nothing else crosses: no namespace, task queue, workflow type or retry settings.



DEMO STEP 8 OF 9 · NEXUS

Four failures, one boundary

Support and payments each own a namespace. Nexus is the one call between them; four failures hit it mid-flight.

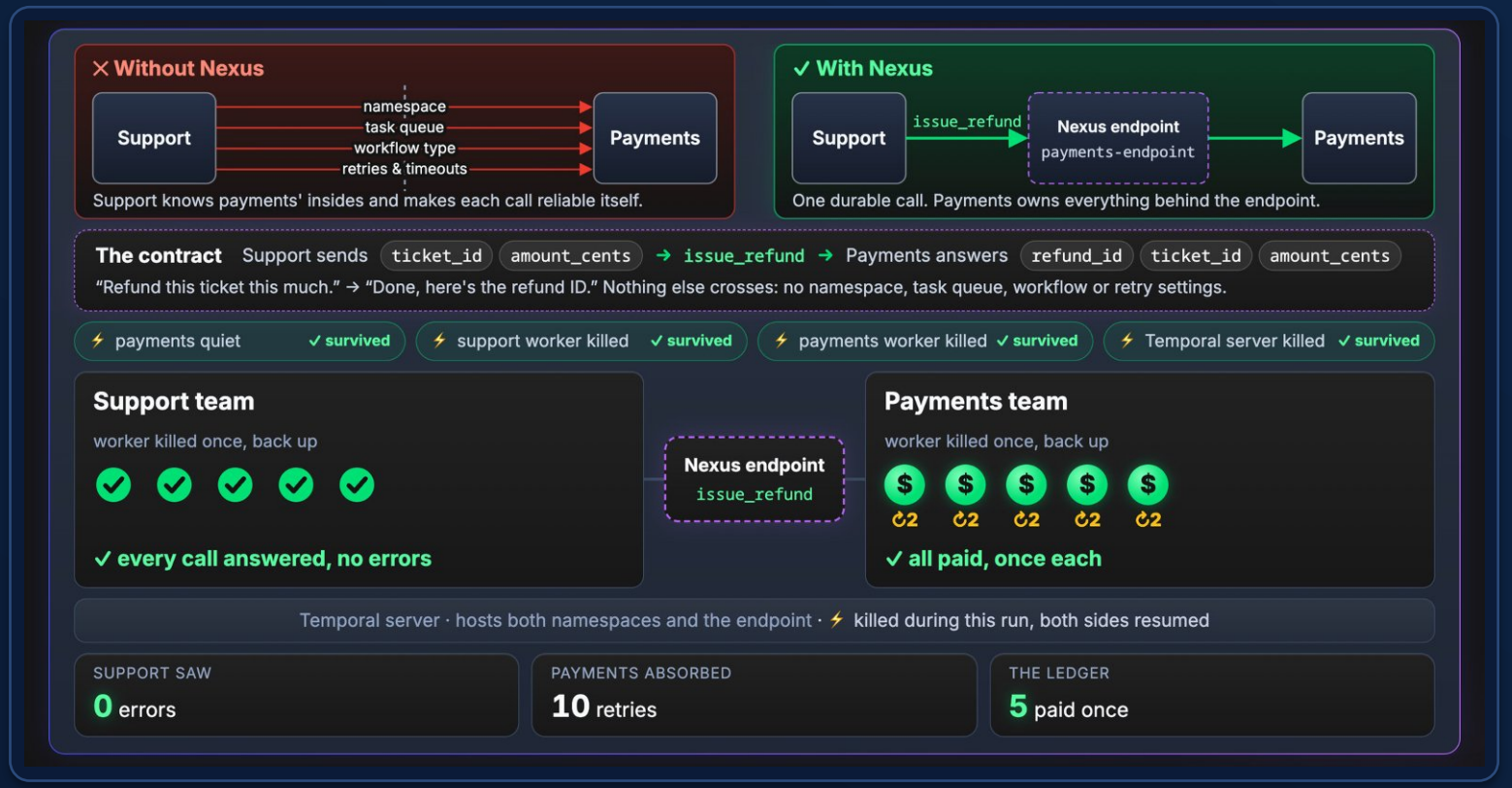
WATCH

Support never sees an error. Payments does the retrying.

RESULT

Support saw no errors. Payments retried with the same key. Each refund paid once.

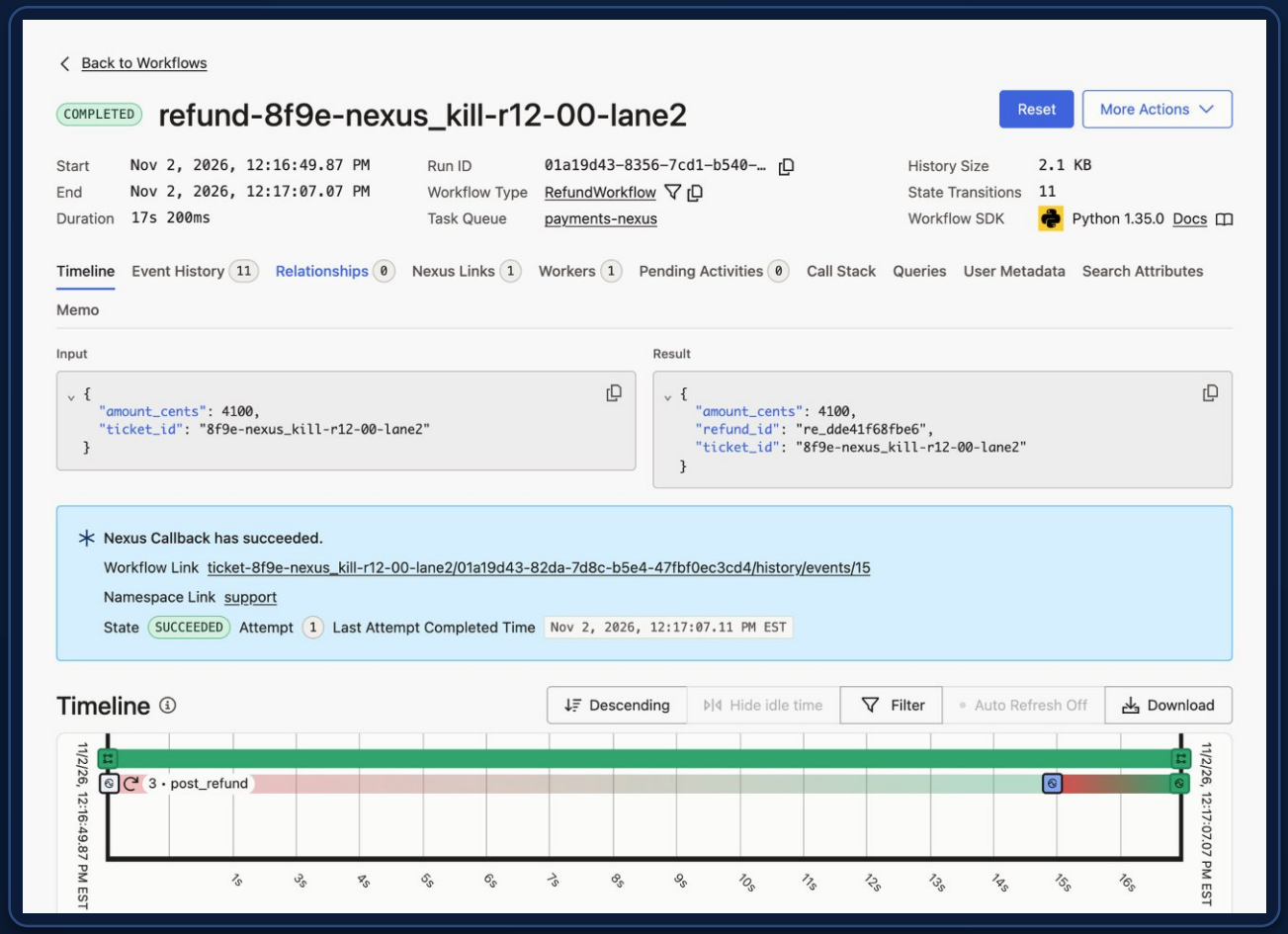
In this capture: Support saw 0 errors. Payments absorbed 10 retries. The ledger booked 5 refunds, each once.



▶ Click the picture to play the clip in PowerPoint or Keynote.

WHAT PAYMENTS SAW

Three attempts, one refund: the retries live inside payments



The Temporal Web UI, payments namespace, from the capture run. Local dev server.

Attempt 1 gave up at the 3-second client timeout while payments held its reply. The refund was already booked.

Attempt 2 hit its 8-second start-to-close timeout when its worker was killed.

Attempt 3 succeeded. Attempts 2 and 3 sent the same key, so the ledger answered from the stored refund.

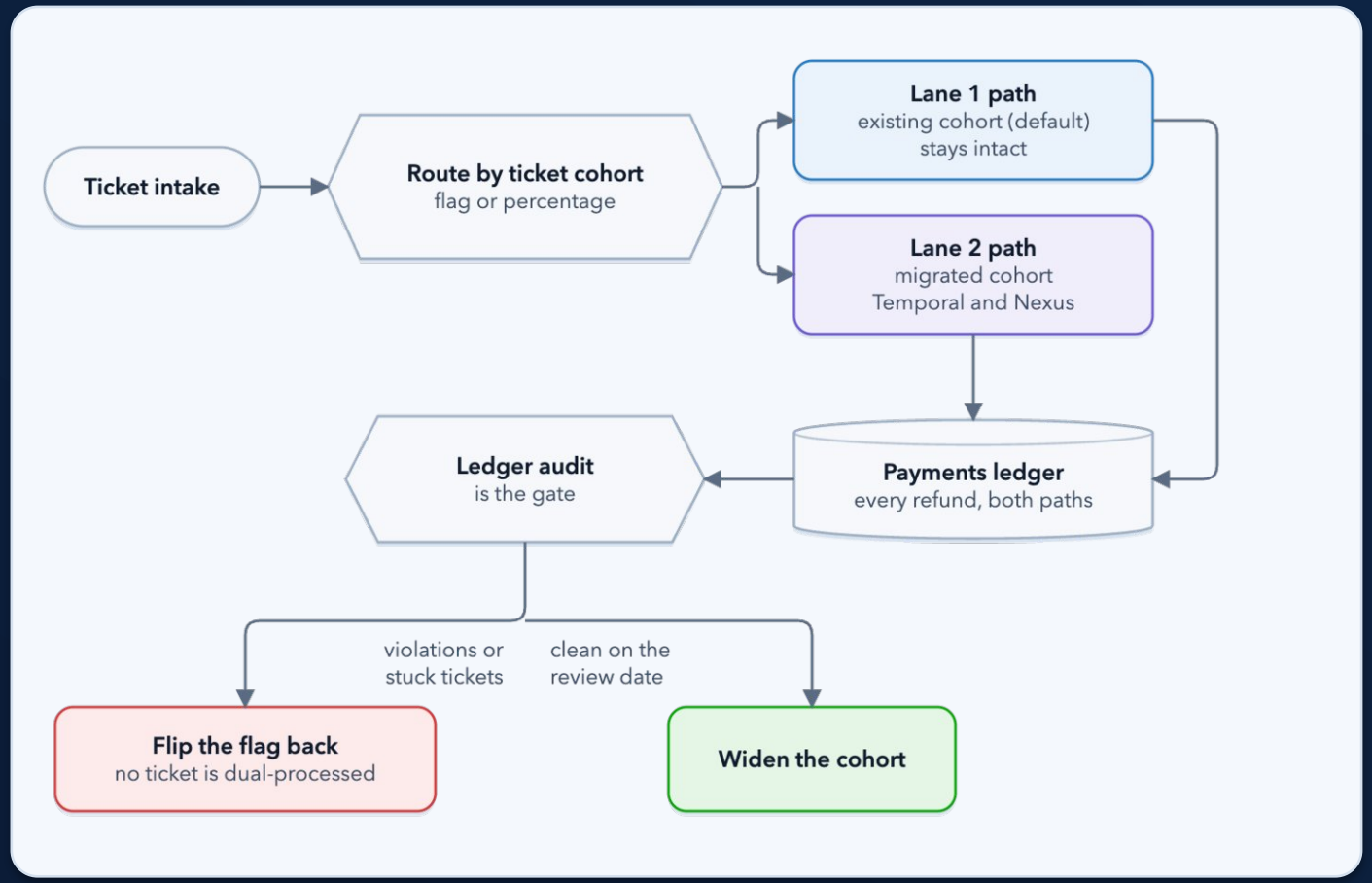
WHAT SUPPORT SAW

Its Nexus operation stayed started, at attempt 1, with no error the whole time. Retries inside the handler's workflow are invisible to the caller.

RUNNING THE TRANSITION

Migrate one process, beside the old one

- **Route by cohort.** A flag or a percentage of a hash of the ticket ID, so each ticket's path is stable.
- **Gate on the ledger.** A scheduled audit compares approver records with the payments ledger and never reads either path's own state.
- **Roll back by flipping the flag.** New tickets return to the old path. Tickets already on Temporal finish; stop sending new work, do not kill the workers.
- **Keep the old path.** Do not decommission it until the review date has passed.



The plan, not a measurement. The ledger audit exists in the repository; the router and the flag do not.

JUDGING THE PILOT

Judge the pilot, then expand, hold or roll back

At the review date	Decision
Zero duplicates and zero unapproved refunds in the migrated cohort; no stuck tickets past the threshold; approval-to-refund time no worse than baseline; operating load within what the teams agreed	Expand. Widen the cohort, then add the next action (credits, partial refunds)
Invariants clean, but operating load over the limit, or payments not yet comfortable owning the handler	Hold. Fix the operating model before widening
Any duplicate or unapproved refund in the migrated cohort that traces to the new design, or a stuck ticket the runbook cannot resolve	Roll back new tickets and find the cause before retrying

MEASURE, PER COHORT

- Ledger-audit violations: duplicate refunds, refunds without an approval, approved tickets with no refund.
- Stuck tickets: approved more than N hours ago with no refund, N from the existing path's approval-to-refund time.
- Time from approval to refund, and the length of approval waits that survived a deploy.
- Operating load: pages per week, time spent on workflow incidents, runbook steps used.

The thresholds are proposals to agree between the teams (expected), not measurements.

DEMO STEP 9 OF 9 · CONCLUSION

Delivered at least once, paid exactly once



THE RECOMMENDED PATH, IN ORDER

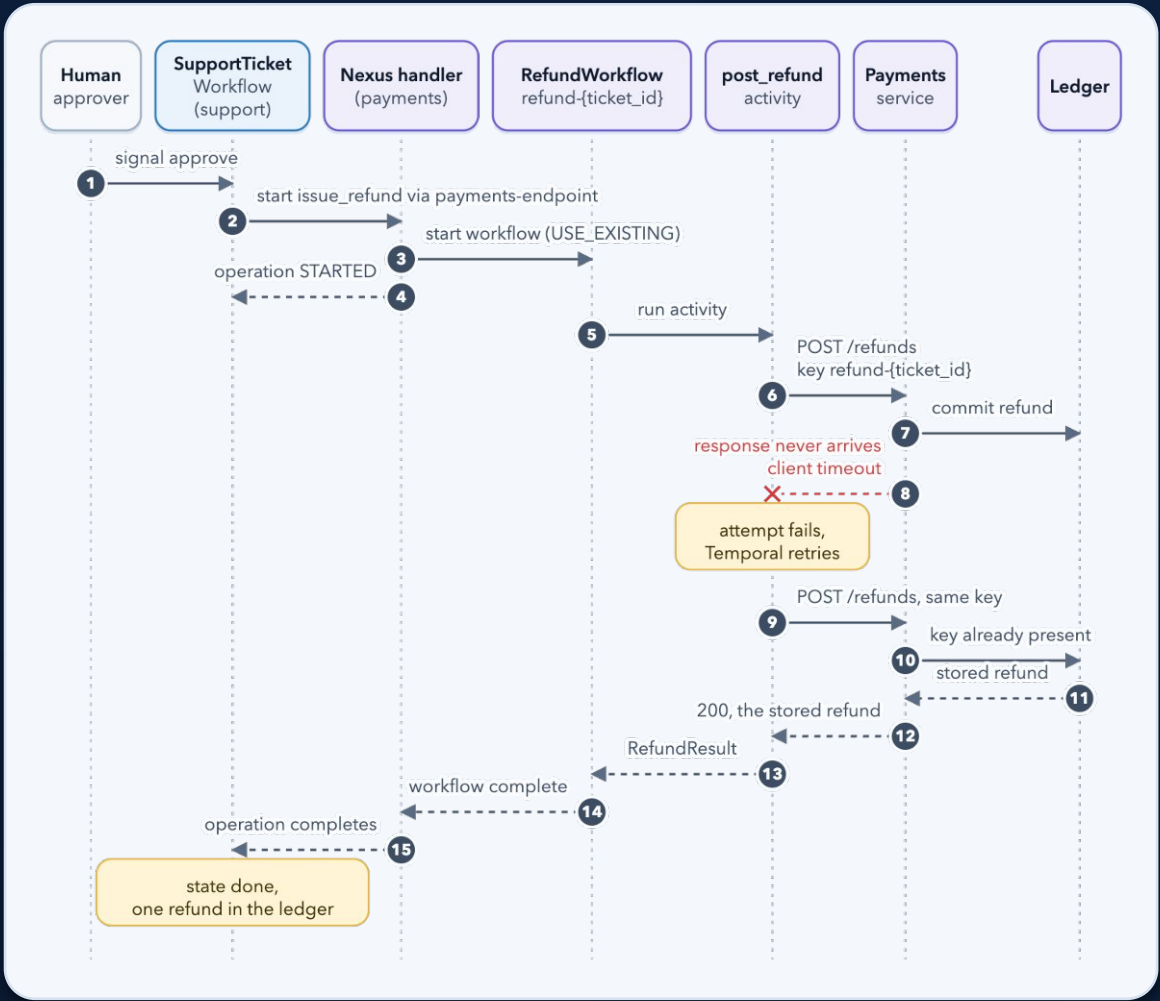
1. **This week:** derive the key as refund-<ticket_id> and run a ledger audit over your existing path.
2. **First Temporal step:** refund approval only, inside one team's namespace, with no Nexus yet.
3. **Nexus when the second team is ready:** payments publishes the contract and takes over the handler, the retries and the key.
4. **Expand by pattern:** other agent actions that need approval and cross a team boundary.

A key per attempt pays twice.

A key per refund pays once.

Pick one call in your system and make it paid once.

One refund, message by message, with the lost reply



Messages 5 to 8. Payments commits the refund, then the reply is lost. The activity attempt fails at the client timeout.

Messages 9 to 13. The retry carries the same key. The ledger already has it and returns the stored refund.

Messages 14 and 15. The workflow completes and the operation completes. One refund is in the ledger.

What the caller sees. The operation stays STARTED until it completes. It never sees the retry.

APPENDIX B · EVIDENCE AND LIMITS

What each claim rests on, and what this evidence does not show

Tag	Meaning
MEASURED	Scored by the ledger audit in a reproducible run
OBSERVED	Seen by inspecting Temporal or the code; not scored
DOCUMENTED	Stated in Temporal's documentation
EXPECTED	My reasoning; not built or tested
ASSUMPTION	Needs a real customer's numbers to confirm

WHAT THIS EVIDENCE DOES NOT SHOW

- **A rate.** A handful of runs is a demonstration, not a sample; the existing stack's counts vary by a ticket or two.
- **Your system.** The existing stack here is a model of a competent team's. Your own baseline decides the urgency.
- **That durable execution alone made Temporal safe.** The stable-key runs show the existing stack reaches zero duplicates with the same kind of key.
- **Scale or latency.** Local processes and SQLite on one machine model crash and retry semantics only.
- **Not built:** cancellation, the Cloud allowlist, a production router, endpoint retargeting with operations in flight.

APPENDIX C · REPRODUCE THE RESULTS

Run it yourself on macOS, Linux or Windows (WSL2)

```
git clone https://github.com/agileresearchservices/paid-once.git && cd paid-once
make install
make scenario   # both paths, stub model, about four minutes, no API key
PAIDONCE_L1_STABLE_KEY=1 make scenario   # the existing stack with the key from the ticket
make demo   # visual walkthrough with a live model (needs ANTHROPIC_API_KEY)
```

docs/architecture.md

The design, and why the existing stack is a fair comparison.

docs/scenario-results.md

What make scenario prints and how to read it.

docs/development.md

A code tour: the engine, the audit and the snapshot contract.

An independent case study, prepared for Temporal as part of a panel interview. It was not produced by Temporal. Temporal is a trademark of Temporal Technologies, Inc.